

KittyCatQuant

COMS 4020: Computer Science Senior Project

Team Members

Caleb Moe

Peter Collins

Kennedy Wendl

Dhvani Mistry

Client

Dr. Hung Phan

Instructor

Dr. Simanta Mitra

Teaching Assistant

Haniyeh Fekrmandi

Table of Contents

Table of Contents.....	1
Chapter 1: Requirements.....	4
1.1 Introduction.....	4
Problem Statement.....	4
1.2 Scope.....	4
Acceptance Criteria.....	5
Milestones.....	6
Limitations.....	6
1.3 Functional Requirements.....	7
1.4 Non-Functional Requirements.....	8
Chapter 2: Design.....	9
2.1 Overview.....	9
2.2 System Architecture.....	10
2.3 API Design.....	11
Main API Modules.....	11
1. Authentication API.....	11
2. User API.....	11
3. Portfolio and Trade API.....	11
4. Portfolio Performance API.....	12
5. Portfolio Snapshot API.....	12
6. Watchlist API.....	12
7. Watchlist Prices API.....	12
8. Symbol and Market Data API.....	13
9. Articles API.....	13
10. Chat API.....	13
API Response Format.....	14
Design Considerations.....	14
2.4 UI Design.....	14
Login and Authentication Interface.....	15
Dashboard and Navigation Layout.....	16
Watchlist Interface.....	17

Learn Mode.....	17
Buy and Sell Interface.....	18
Settings Interface.....	19
Stock News Interface.....	20
AI Chat Interface.....	21
Chapter 3: Work Completed.....	22
2.1 Caleb Moe.....	22
2.2 Peter Collins.....	23
2.3 Kennedy Wendl.....	27
2.4 Dhvani Mistry.....	29
Chapter 4: Results.....	31

Chapter 1: Requirements

1.1 Introduction

KittyCatQuant is an interactive web-based stock trading platform designed and developed by a team of four undergraduate students at Iowa State University enrolled in COMS 4020: Computer Science Senior Project. The platform aims to provide a safe environment where young adults can learn more about the stock market, practice trading, create and build strategies, as well as receive trading advice without fear of losing money.

The application combines a modern React frontend with a .NET backend that exposes a REST API, backed by a PostgreSQL database managed via Entity Framework Core and documented with Swagger. The system integrates several external services, including a stock data provider, a news API, and an OpenAI language model, and supports user authentication, simulated portfolio management, real-time market data visualization, AI-driven market insights, and an educational LearnMode designed to improve understanding for beginner traders.

Problem Statement

A majority of students and young adults lack foundational knowledge of the stock market and stock market trading. This gap exists due to a lack of educational resources and financial risk involved with trading platforms. Therefore, there is a clear need for a platform that is accessible, interactive, and risk-free.

1.2 Scope

KittyCatQuant's target audience is college students and young adults. Specifically, those with little to no knowledge of stock trading or who have minimal exposure. The platform is delivered as a web application and can be accessed via modern browsers. The scope of the project encompasses the following areas:

- Simulated stock trading (buy and sell orders with virtual funds)
- Real-time and historical market data visualization
- Portfolio tracking with daily valuation snapshots
- Personalized watchlists and stock search
- AI-powered chatbot for market education and personalized Q&A
- Stock news aggregation with sentiment and impact analysis
- An opt-in LearnMode for contextual UI guidance
- Secure user account management

However, the platform does not execute trades on live markets, provide licensed financial advice, or manage real currency. KittyCatQuant is intended solely as an educational simulation tool.

Acceptance Criteria

The following criteria define the conditions KittyCatQuant will be considered complete and accepted:

1. Users can register and manage account settings securely.
2. Users can search for stocks, view historical price charts across multiple time ranges, and make informed decisions from stock reports.
3. Users can execute buy and sell orders that update portfolio holdings and available cash in real time.
4. Users can create and manage watchlists with live market data.
5. Users can view recent news with sentiment analysis, favorite articles, search by keywords, and filter articles by category.
6. Users receive prompt AI chatbot responses to stock market and portfolio questions, with chat history persisted across sessions.
7. Users can toggle LearnMode via settings to display contextual tips and descriptions throughout the application.

Milestones

KittyCatQuant was built around three progressive phases, each phase highlighting different stages of the project. Each demo was spread out over four to five weeks, during which the team presented their work.

Demo 1: Core Framework

- User authentication and account creation
- Basic portfolio and trade execution
- Initial stock search and market data integration
- Stocks watchlist with daily values

Demo 2: Feature Development

- Portfolio tracking and interactive chart visualization
- Stock news aggregation with sentiment analysis and favoriting
- AI chatbot with message history and multithread support
- Expanded stock data and stock history
- User settings and LearnMode

Demo 3: Polish and Finetune

- LLM tool integration with platform features
- Evaluation of LLM and prompt engineering
- Stock buy or sell recommendation
- Stock split handling

Limitations

Simulated Trading: The platform does not interact with or handle any real financial transactions.

Sentiment Analysis: News sentiment analysis may not accurately reflect the true impact on the stock market and is intended for educational illustration only.

Limited Budget: The team utilized free online resources for their source of data. With an increased budget, the platform could be expanded to include more information about the stock tickers and a wider range of news articles.

1.3 Functional Requirements

User Authentication: Users should be able to register, log in, and log out securely. Authorization should be enforced so that a user can only access their account.

Simulated Stock Trading: Users should be able to buy and sell using mock funds. Trades should be recorded and reflected in the user's portfolio in real time.

Portfolio Tracking and Visualization: The program should calculate and store the daily value of each user's portfolio. Interactive chart will display day-to-day changes.

Watchlist: Users should be able to create, name, and delete personal watchlists.

Stock Market Data: The program should retrieve, store, and synchronize stock price data.

Stock News and Sentiment Analysis: The program should retrieve daily news articles and display a sentiment label. Users should be able to search, add/delete favorites, and categorize articles.

Chatbot: User should be able to ask questions regarding the stock market or their portfolio and receive recommendations. Conversations should be stored and organized based on date.

LearnMode: The program should provide an optional LearnMode that overlays tips and descriptions of features.

1.4 Non-Functional Requirements

Usability: KittyCatQuant should be easy for beginner investors to use. The interface should be clear, organized, and supported by LearnMode explanations.

Performance: The application should respond quickly when users log in, view portfolios, search stocks, place trades, or load news. Cached data should be used when possible to reduce delays.

Reliability: The system should update stock data, portfolio snapshots, and backfill data automatically. It should also handle missing external data without crashing.

Security: User accounts should be protected with secure authentication and password hashing. Users should only be able to access their own portfolios, watchlists, chats, settings, and saved articles.

Maintainability: The codebase should be organized into clear frontend components and backend controllers, services, repositories, models, and DTOs. Swagger should help document and test the API.

Chapter 2: Design

2.1 Overview

KittyCatQuant is intended to be a full-stack stock trading simulation platform providing a combination of real-time market data, portfolio management, AI-supported learning, and financial news. It can be seen as an educational tool allowing the user to gain experience in stock market investments and trading while getting acquainted with investment basics.

The app was designed following the client-server architecture, comprising three main components:

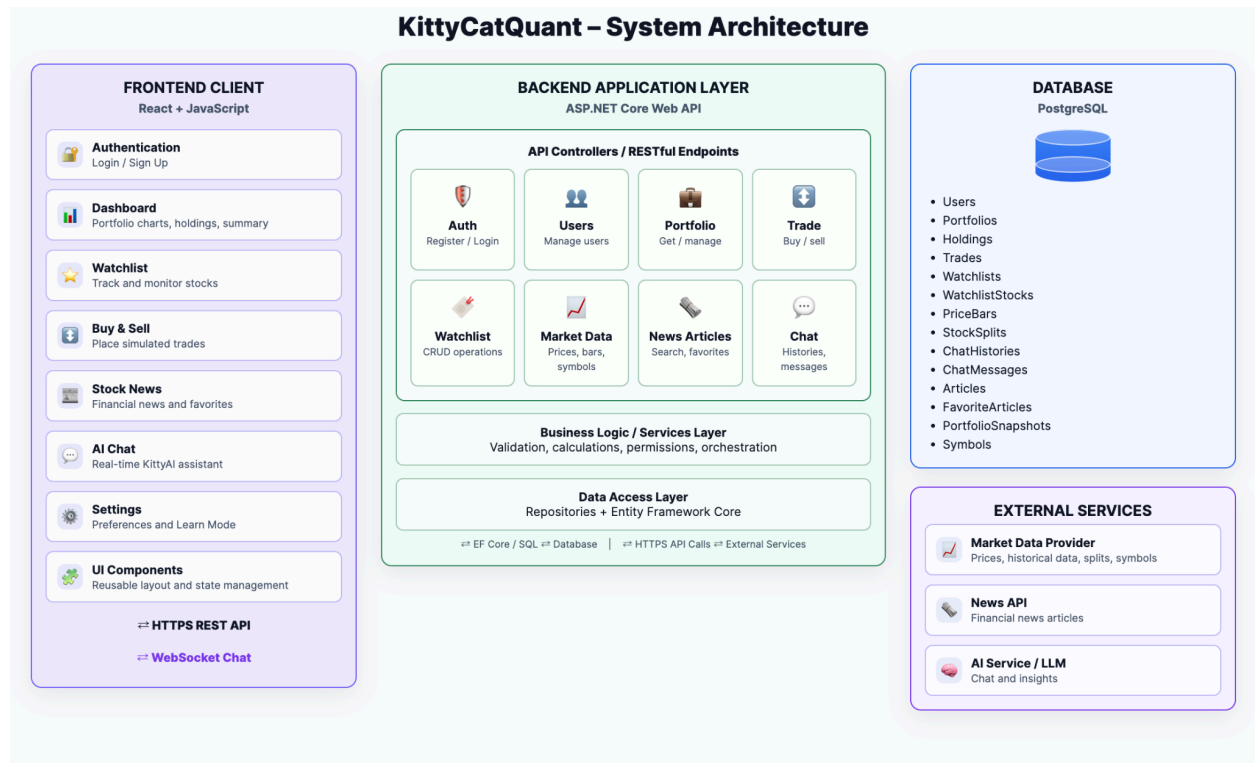
1. Front-end user interface
2. Back-end API and business logic
3. Database and third-party APIs

The frontend was developed to have the dashboard interface, enabling easy navigation between portfolio management, watchlist management, stock trading, financial news, and AI chat features. Back-end REST APIs handle portfolio management, authentication, stock trading, watchlists management, articles, and chat features.

Modularity was one of the goals set during the design process. It implied that all major functionality was split into separate modules and back-end controllers which were supposed to increase its flexibility as well as maintainability of the app.

The system additionally includes AI-powered features such as predictive market insights and chatbot communication to support the platform's educational focus. The "Learn Mode" feature was specifically designed to help users better understand financial concepts, trading workflows, and dashboard terminology.

2.2 System Architecture



2.3 API Design

The StockTraderBackend exposes a REST API documented through Swagger UI using OpenAPI 3.0. The API is organized by feature area, making it easier for the frontend to interact with backend services such as authentication, portfolios, trades, watchlists, market data, articles, and chat history.

Main API Modules

1. Authentication API

The authentication endpoints allow users to register and log in.

1. POST `/auth/register` creates a new user account using username, password, and email.
2. POST `/auth/login` authenticates a user using either username/password or email/password.

2. User API

The user API supports basic user management.

1. GET `/users` retrieves all users.
2. PUT `/users/{userId}` updates an existing user.
3. DELETE `/users/{userId}` deletes a user.

3. Portfolio and Trade API

The portfolio API manages user investment portfolios.

1. GET `/portfolio` retrieves all portfolios.
2. GET `/portfolio/{userId}` retrieves a portfolio for a specific user.
3. DELETE `/portfolio/{portfolioId}` deletes a portfolio.
4. POST `/trade` executes a buy or sell trade.

The trade endpoint validates the request, checks cash or share availability, fetches the current stock price, updates holdings, and records the trade.

4. Portfolio Performance API

The performance API calculates the current value and gain/loss of a portfolio.

1. GET /api/portfolios/{portfolioId}/performance

This endpoint returns cash balance, holdings value, total portfolio value, cost basis, unrealized gain/loss, gain/loss percentage, top gainer, top loser, and position-level performance.

5. Portfolio Snapshot API

The snapshot API stores and retrieves historical portfolio values.

1. POST /api/portfolio/snapshot/{date}
creates portfolio snapshots for a trading date.
2. GET /api/portfolio/snapshot/{portfolioId}
retrieves historical portfolio snapshots.

6. Watchlist API

The watchlist API allows users to create and manage stock watchlists.

1. GET /api/watchlists retrieves all watchlists for a user.
2. POST /api/watchlists creates a new watchlist.
3. GET /api/watchlists/{id} retrieves a specific watchlist.
4. DELETE /api/watchlists/{id} deletes a watchlist.

Watchlist stock endpoints support adding, viewing, and removing symbols from a watchlist.

7. Watchlist Prices API

The symbol API provides access to supported stocks.

1. GET /api/watchlists/{watchlistId}/prices

This endpoint returns a watchlist with current stock prices for each stock. It also verifies that the requesting user owns the watchlist.

8. Symbol and Market Data API

1. GET /api/symbols retrieves all supported symbols.
2. GET /api/symbols/ticker/{ticker} retrieves stock information by ticker.
3. GET /api/symbols/company searches for a stock by company name.
4. GET /api/symbols/company-names retrieves all company names
The price bars endpoint retrieves historical daily stock data:
5. GET /api/market/bars

9. Articles API

The articles API provides financial news.

1. GET /api/articles/search searches news articles by query.
2. GET /api/articles/headlines retrieves top headlines.
3. POST /api/articles/favorites/{articleId} favorites an article.
4. DELETE /api/articles/favorites/{articleId} removes a favorite.
5. GET /api/articles/favorites retrieves a user's favorite articles.

These endpoints allow users to browse market-related news and save articles.

10. Chat API

The chat API stores user chat sessions and messages.

1. GET /api/chat/{userId} retrieves all chat histories for a user.
2. POST /api/chat/{userId} creates a new chat session.
3. GET /api/chat/{userId}/{chatId} retrieves a specific chat.
4. PUT /api/chat/{userId}/{chatId} updates the chat session name.
5. DELETE /api/chat/{userId}/{chatId} deletes a chat history.

API Response Format

Most endpoints return JSON responses. Successful responses include domain-specific objects such as User, PortfolioDTO, ArticleResponseDto, Watchlist, PriceBarsResponse, and PortfolioPerformanceSummaryDto.

Error responses commonly use the ProblemDetails format, which includes:

```
json
{
  "type": "string",
  "title": "string",
  "status": 0,
  "detail": "string",
  "instance": "string"
}
```

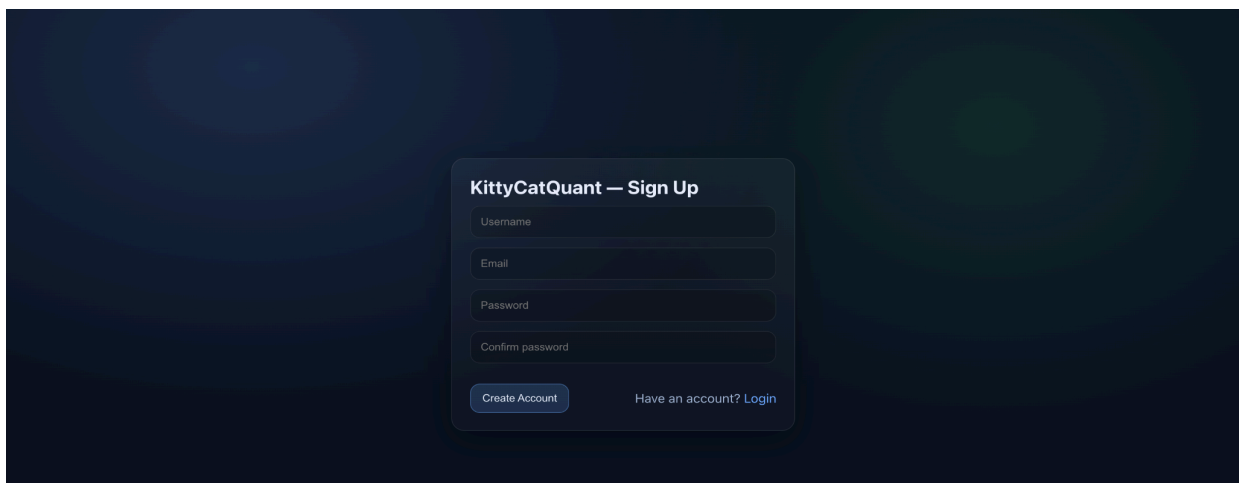
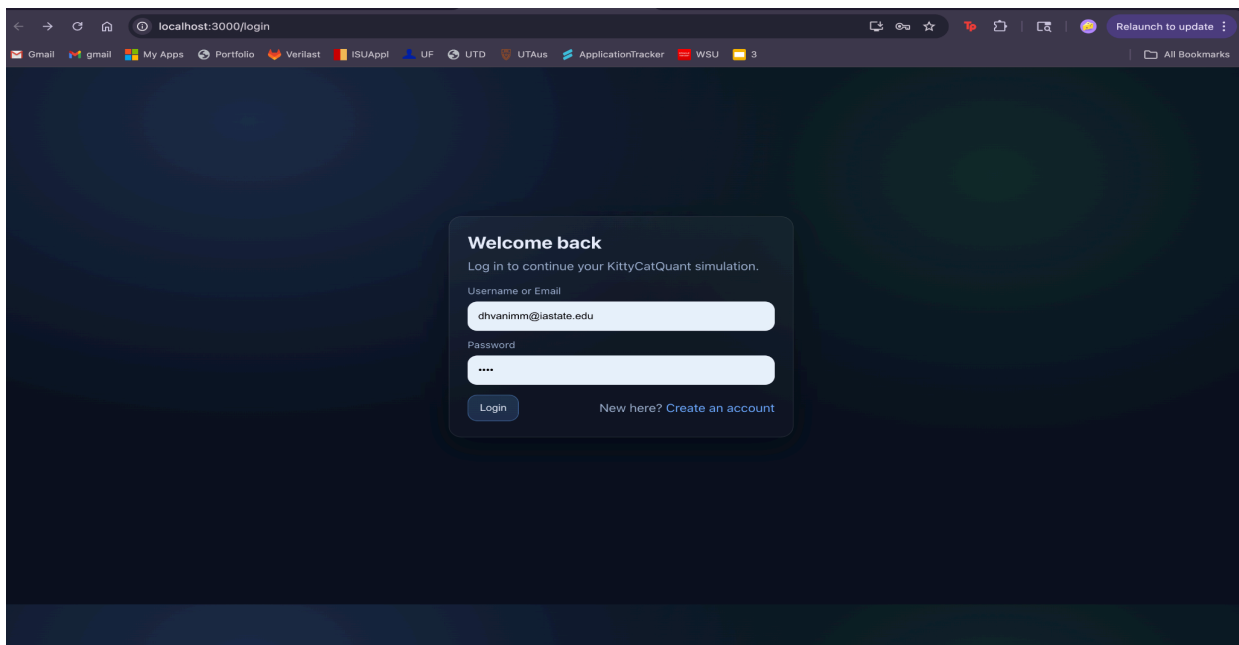
Design Considerations

The API was designed around clear separation of backend responsibilities. Each major application feature has its own controller group, which improves maintainability and makes the Swagger documentation easier to navigate. Swagger UI was especially useful for testing endpoints, checking request/response schemas, and validating how the frontend should communicate with the backend.

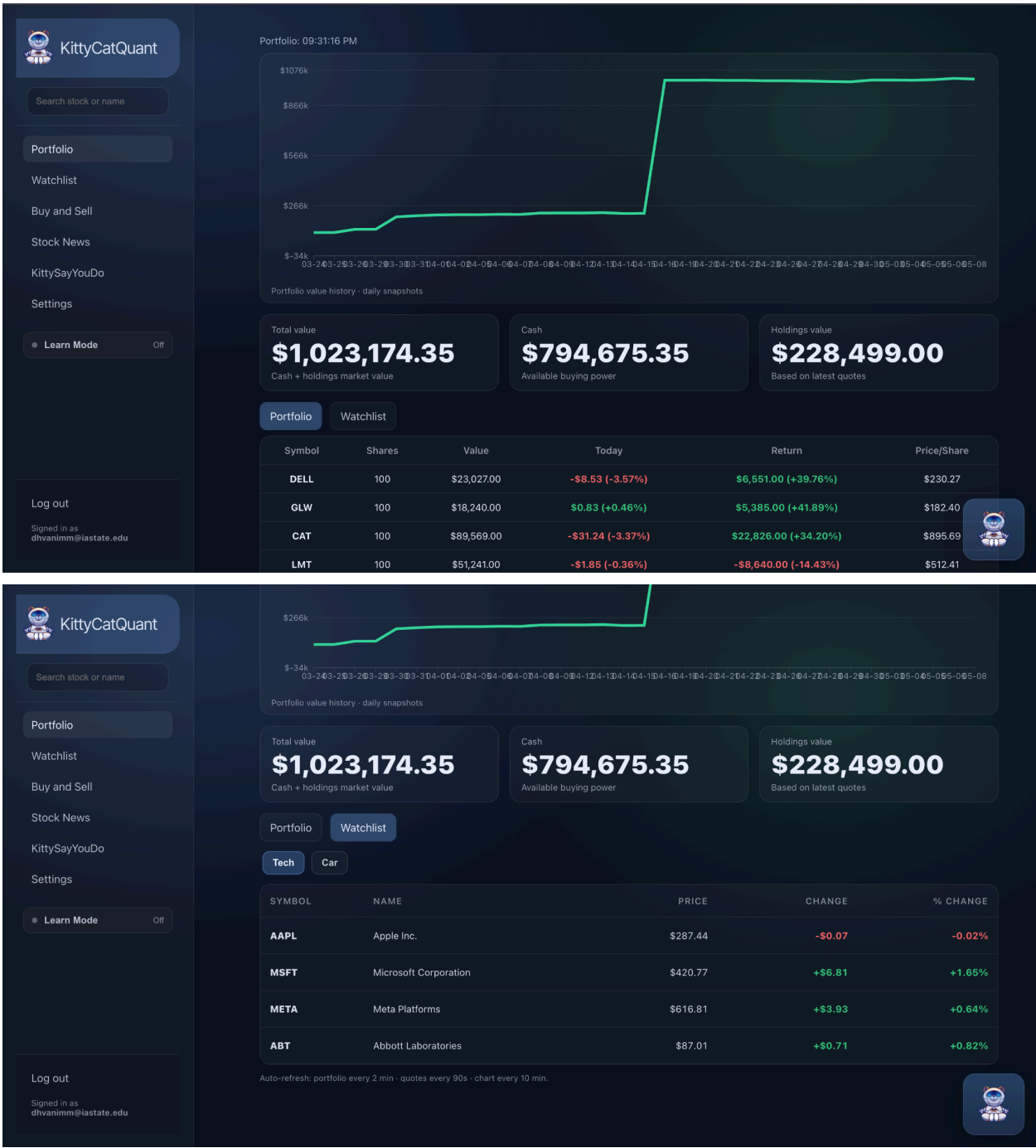
2.4 UI Design

The KittyCatQuant user interface was designed to provide a modern, interactive, and intuitive stock trading experience. The frontend uses a dark-themed dashboard layout with responsive navigation, real-time financial data visualization, and AI-assisted trading features. The design focuses on usability, readability, and fast access to important trading information. Login and Authentication Interface

Login and Authentication Interface



Dashboard and Navigation Layout



Watchlist Interface

The screenshot shows the 'Watchlist' section of the KittyCatQuant application. On the left is a sidebar with navigation links: Portfolio, Watchlist (selected), Buy and Sell, Stock News, KittySayYouDo, and Settings. At the bottom of the sidebar is a 'Learn Mode' toggle set to 'Off' and a 'Log out' button. The main content area is titled 'Watchlist' with the subtitle 'Manage your watchlists and tracked stocks.' It features a 'New watchlist name' input field with a 'Create' button. Below this is a 'Your Watchlists' section showing two watchlists: 'Tech' and 'Car', each with a 'Delete' button. The 'Tech' watchlist is expanded, showing '4 stocks • 3 up • 1 down'. It includes a search bar with the text 'Search stock (e.g. AAPL or Apple)' and an 'Add Stock' button. A table lists the stocks in the watchlist:

SYMBOL	NAME	MARKET PRICE	CHANGE	% CHANGE
AAPL	Apple Inc.	\$287.44	-\$0.07	-0.02%
MSFT	Microsoft Corporation	\$420.77	+\$6.81	+1.6%
META	Meta Platforms	\$616.81	+\$3.93	+0.6%

At the bottom right, there is a small floating window showing a mobile app interface with a cat icon.

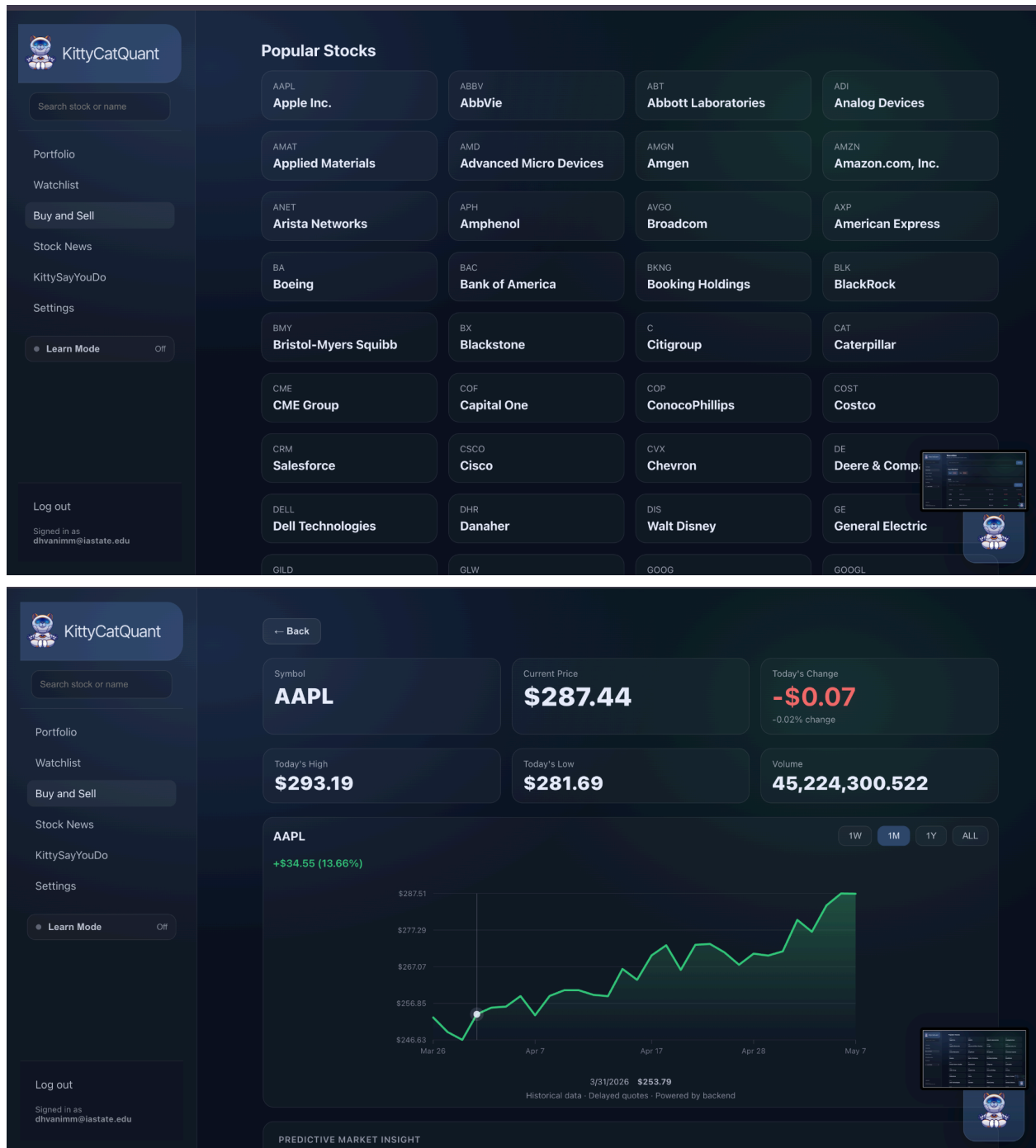
Learn Mode

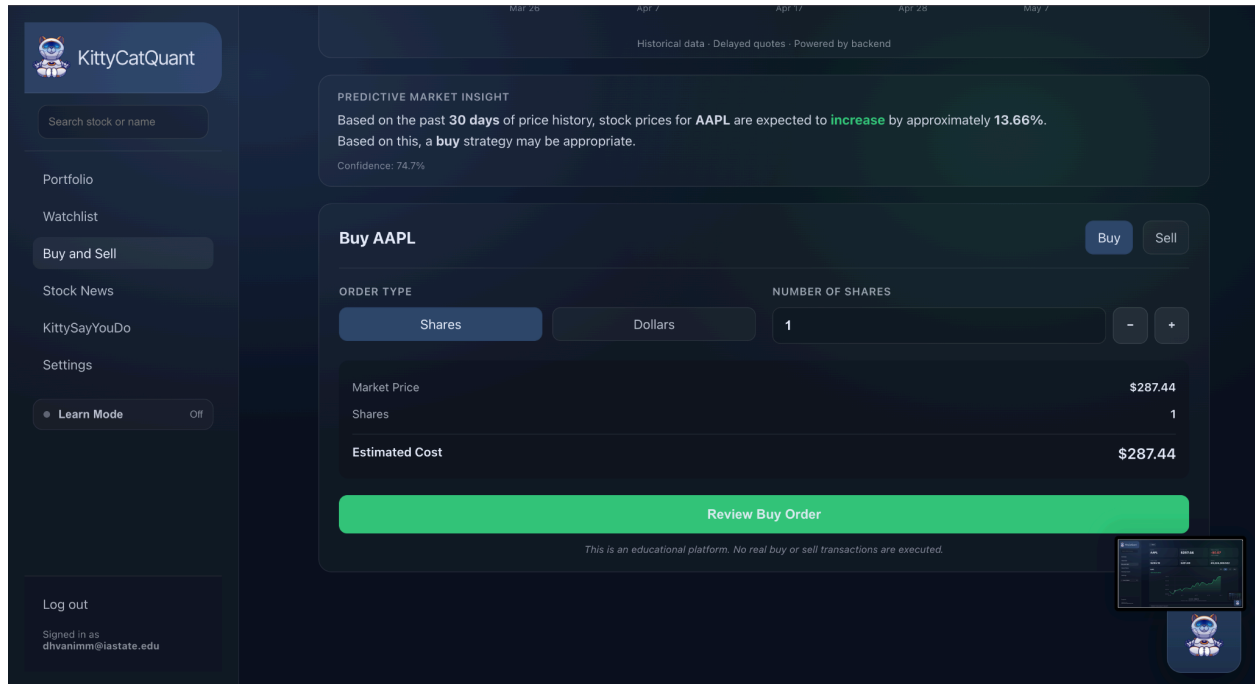
The screenshot shows the 'Learn Mode' section of the KittyCatQuant application. The sidebar on the left has navigation links: Stock News, KittySayYouDo, and Settings. The 'Learn Mode' toggle is set to 'On'. The main content area displays a 'Total value' section with a large number: '\$1,023,174.35'. Below this, it says 'Cash + holdings market value'. There are two buttons: 'Portfolio' and 'Watchlist'. Below these buttons is a table showing the portfolio holdings:

Symbol	Shares	Value
DELL	100	\$23,027.00

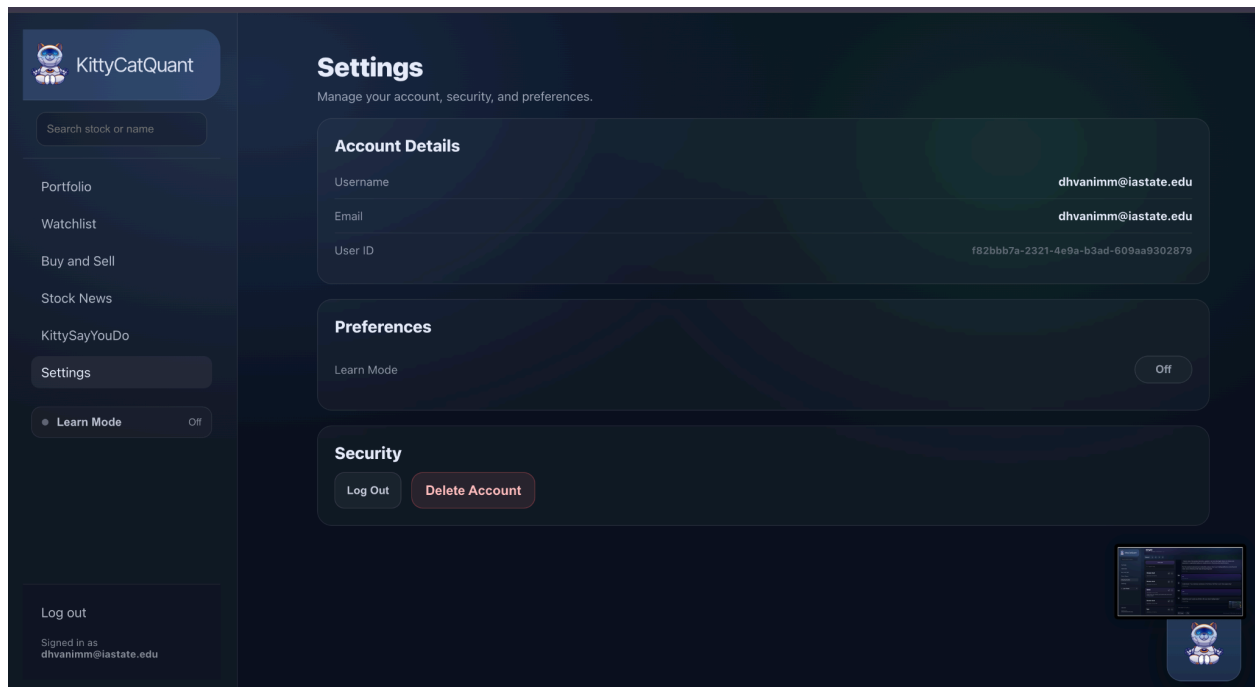
A tooltip is visible over the 'Total value' section, stating: 'Total value is the combined value of your cash plus the current market value of all stocks you hold.'

Buy and Sell Interface





Settings Interface



Stock News Interface

KittyCatQuant

Search stock or name

Portfolio

Watchlist

Buy and Sell

Stock News

KittySayYouDo

Settings

Learn ModeOff

Log out

Signed in as
dhvanimm@iastate.edu

Search articles by headline

AP News · 5/8/2026, 9:00:00 AM

Trump administration settles meatpacking antitrust case

Sentiment: BEARISHImpact: MARKETConfidence: 81%

Read More

CNBC · 5/8/2026, 8:59:55 AM

Fitness wearable Whoop to offer on-demand clinician access in U.S.

Sentiment: BULLISHImpact: MARKETConfidence: 78%

Read More

TechCrunch · 5/8/2026, 8:00:00 AM

The biggest US power grid is under strain from AI - and no one is happy

Sentiment: BEARISHImpact: MARKETConfidence: 88%

Read More

Porsche Newsroom · 5/8/2026, 7:20:40 AM

Porsche sharpens focus on core business

KittyCatQuant

Search stock or name

Portfolio

Watchlist

Buy and Sell

Stock News

KittySayYouDo

Settings

Learn ModeOff

Log out

Signed in as
dhvanimm@iastate.edu

Stock News

Latest Headlines

Search articles by headline

TechCrunch · 5/8/2026, 8:00:00 AM

The biggest US power grid is under strain from AI - and no one is happy

Sentiment: BEARISHImpact: MARKETConfidence: 88%

Read More

CNBC · 5/8/2026, 8:59:55 AM

Fitness wearable Whoop to offer on-demand clinician access in U.S.

Sentiment: BULLISHImpact: MARKETConfidence: 78%

Close

Wearable fitness tracker Whoop announced on Friday it will introduce in-app access to on-demand licensed clinicians for users in the United States. The new feature comes alongside a suite of health and artificial intelligence-driven features launchin... [1964 chars]

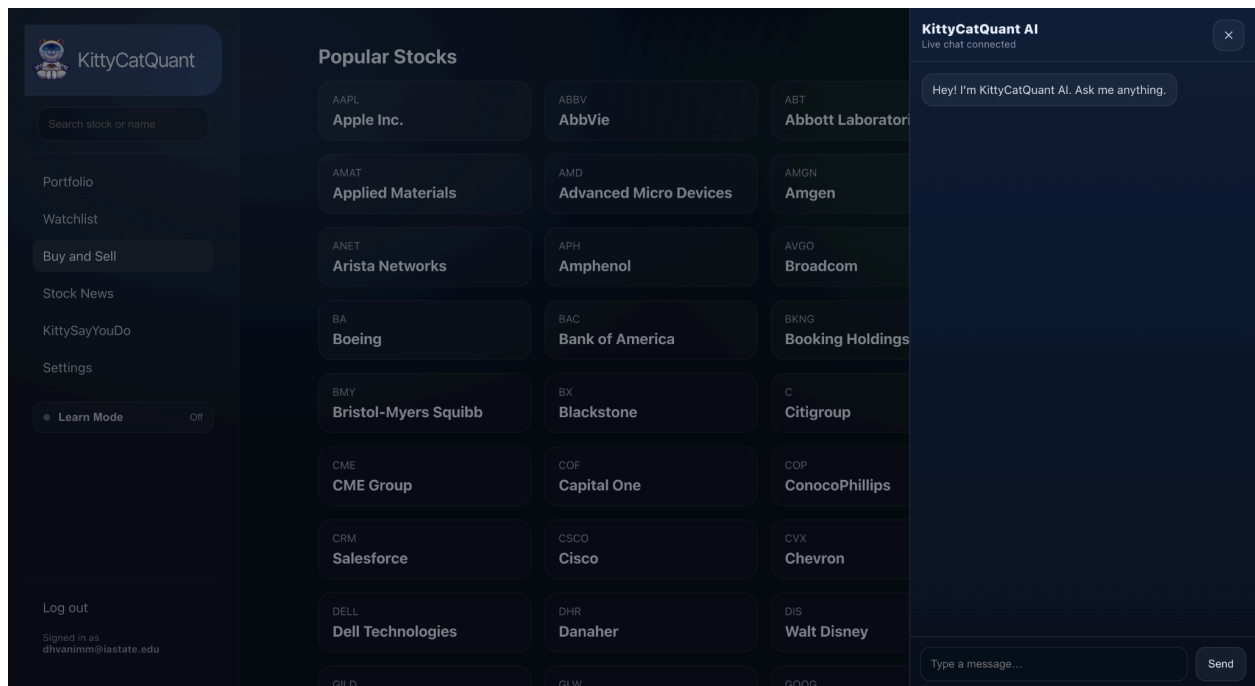
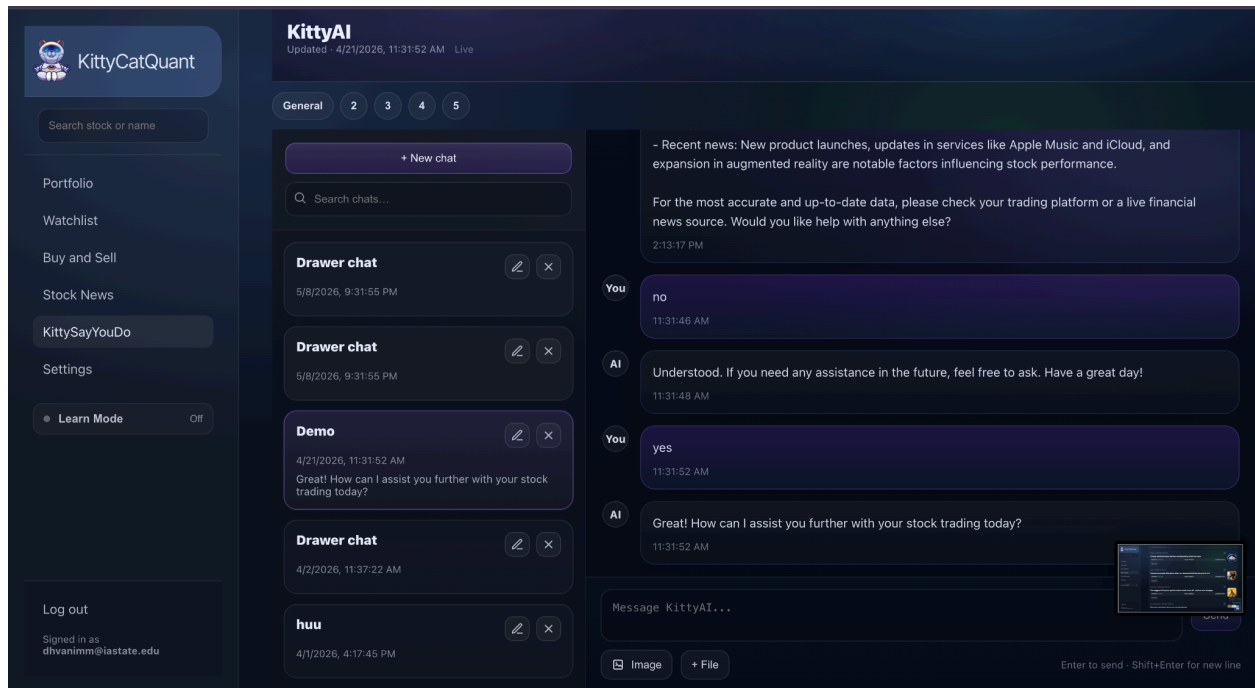
Read full article →

AP News · 5/8/2026, 9:00:00 AM

Trump administration settles meatpacking antitrust case

Sentiment: BEARISHImpact: MARKETConfidence: 81%

AI Chat Interface



Chapter 3: Work Completed

2.1 Caleb Moe

I worked mainly on the backend systems, stock data infrastructure, news logic, and LLM tooling. My work focused on improving the reliability of our data pipelines, reducing unnecessary API calls, and connecting the chatbot to real application data.

I started by helping with backend documentation, setting up the database connection, and adding early support for symbols and price bar data. I then built the price bar controller, which allowed the backend API to expose historical stock price data.

After that, I worked on the news article feature and later refactored the news logic so the system prioritized cached articles before calling the external news API. This helped reduce API usage and avoid rate limit issues.

A major part of my work involved stock data ingestion. I connected the application to Massive for stock data syncing, cleaned up symbol-related logic, and improved the backfill system. I moved toward a rolling two-year backfill approach and added a retention plan to remove outdated price bars.

I also designed and implemented the logic for detecting stock splits. Instead of manually adjusting historical prices, the system detects splits, marks affected stocks for backfill, and refreshes adjusted data directly from Massive.

Later in the project, I created the initial AI service for the chatbot and built the LLM tooling. This allowed the chatbot to access portfolios, symbols, price bars, trade history, watchlists, news articles, favorite articles, stock split data, and trading features. I also created supporting services, such as the portfolio summary service and the watchlist-plus-price service, to make AI responses more reliable.

Near the end of the project, I completed the AI trading feature, allowing the chatbot to help users buy and sell stocks through natural language while still relying on backend services for the actual trading logic.

- Created + Refactored the news service to prioritize cached articles and reduce external API calls.
- Completed the news article favoriting feature.
- Set up the PostgreSQL database.
- Helped initiate Entity Framework adoption.
- Built the price bar logic for storing and retrieving stock price history.
- Created the stock data backfill runner.
- Created the daily stock data runner for updating prices.
- Implemented a rolling two-year stock data retention plan.
- Designed and implemented the stock split service and runner.
- Changed the stock split architecture to avoid double-adjusting already adjusted data.
- Created the initial AI service used by the chatbot.
- Built LLM tools for portfolios, symbols, price bars, trade history, watchlists, news, favorite articles, stock splits, and trading.
- Created a portfolio summary service for AI-friendly portfolio performance results.
- Created a watchlist plus price service.
- Completed the AI trading feature.
- Improved the overall connection between the chatbot and the project's backend services.

2.2 Peter Collins

I worked primarily on the backend systems, specializing in portfolio, trade, watchlist, and user infrastructure. My work early in the semester was centered around providing the necessary scaffolding for the application to take shape and expanding the features to meet our vision for the system. For the second sprint, I worked primarily on establishing chatbot capabilities to fulfill the vision of an AI trading assistant, including sending messages, receiving messages and storing and organizing chat history. After the second sprint when the system as a whole was very developed, I shifted my focus to LLM evaluation techniques.

I started by establishing the basic infrastructure for user creation and access, establishing login and sign up endpoints, and ensuring authentication was established. Next I worked on the portfolio system. A portfolio was created for each user upon user creation, accessible with a user ID. Then I established capability for users to perform trades. Trading is done through a single endpoint for all transaction types.

Trading is done by sending a trade request, which contains several pieces of information: the ID of the portfolio performing the trade, the ticker representing the asset being traded, the quantity of shares being traded, and the trade type, representing whether the asset is being bought or sold. Using this information, the API decides whether or not the trade is legal. If the trade is legal, the trade will be performed and the state of the portfolio after the trade will be returned. If the trade is illegal, a message will be returned explaining why the trade is illegal and it will not be performed. I also established the watchlist feature, which allows users to create curated lists of assets to follow and collect that information into a single place.

For the second sprint, I shifted my focus to the chatbot infrastructure. I first created standard CRUD endpoints for creating a thread of messages. Next I created websocket access for messages to be sent and received. I added message IDs and connected each message to threads to allow them to be organized and sorted by timestamp more easily. Lastly, I hooked it up to the initial AI service setup by Caleb to provide real AI messages.

Another focus of this sprint was to complete some quality of life and bug fix work. This included changing the organization of data in chat messages to align with how the front end utilized it, fixing a salting issue with our password hashing, and some small dependency issues within interactions between different modules of the backend.

The next main feature of the second sprint was portfolio valuation tracking. I set up a runner that would calculate the value of every portfolio every day, ensuring that this calculation happened after the stock data runner collected fresh market data. This calculation consists of summing the total value of each asset held, calculated by multiplying the most recent price by the total number of shares, then I add the

current free cash held to the sum of all asset values. This is stored as a daily snapshot of the value. Lastly I created endpoints to sort this data and organize it into the same format as the price bars to allow the front end to easily display it as a graph.

For the third and final sprint, my focus shifted entirely to the LLM trading assistant chatbot. Our client expressed interest in evaluating the LLM to learn about how it understands the stock market and utilizes the data that it has access to, as well as taking steps to improve the LLM as much as possible. The first step in performing the evaluation is to create a rubric to analyze responses. It centered around asking four main questions about the response:

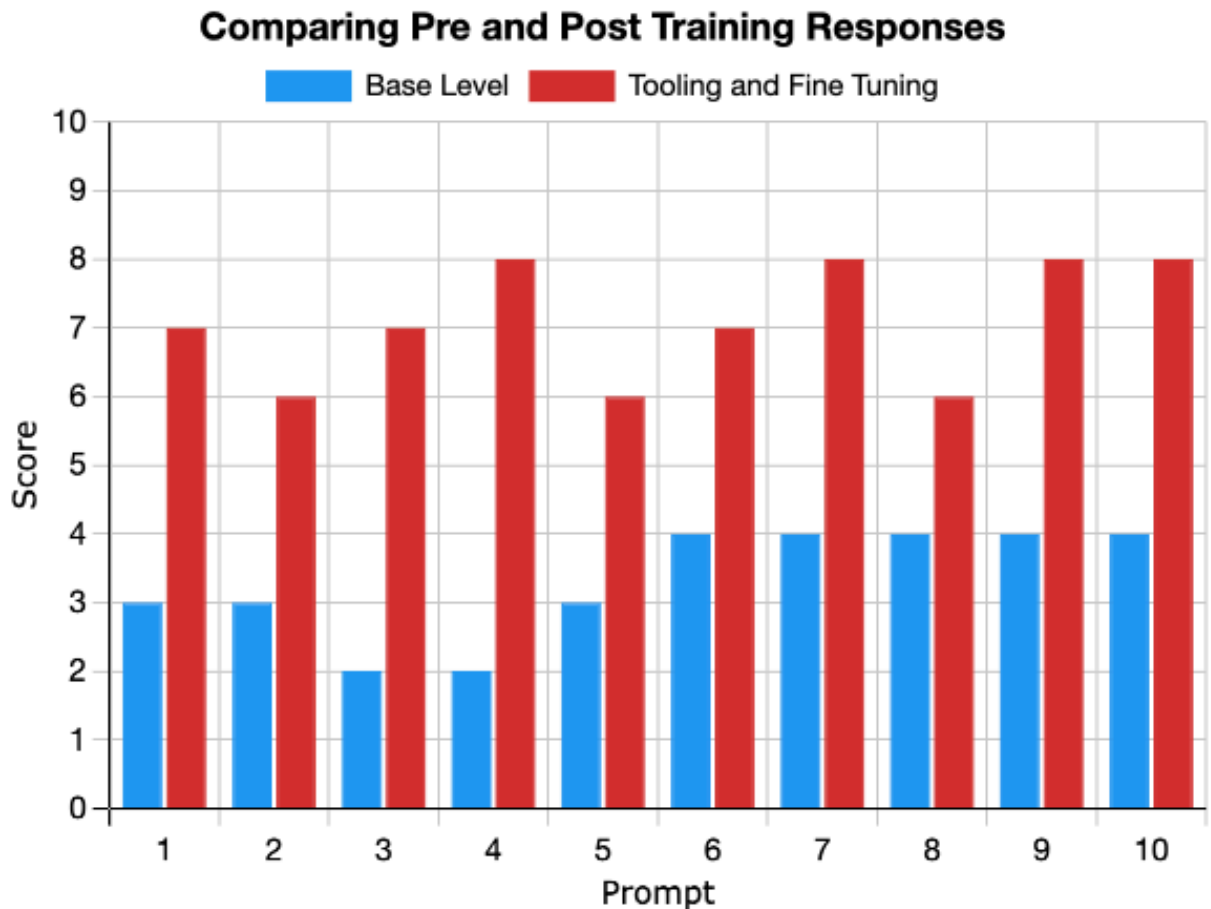
- Does response include items a user can research on their own?
- Does the response include any data or specific evidence?
- Does the response utilize user specific data?
- Does the response answer the proposed question?

After using the chatbot, I found that these questions helped shape a great response, each in a different way. Adding bullet points that can be expanded on is helpful to guide users in a direction of future exploration. Utilizing user or market data helps instill confidence in responses and make them feel personalized. Lastly, answering the question directly makes users feel like the chatbot is actually helpful, rather than avoiding the question or hedging without committing to a recommendation.

This rubric was used to evaluate the response to 10 prompts. The first round of evaluation was used to guide our efforts to improve the LLM's responses. In order to generate higher scores, changes were made to the system prompt and the LLM was given access to the tools created by Caleb with instructions to use them liberally.

After improving the system prompt and granting the LLM access to Caleb's data tools, a second round of evaluation was performed across the same 10 prompts. As shown in the figure above, base-level responses averaged around 3/10, while the tooling and fine-tuned configuration averaged roughly 7/10 — more than doubling the score across nearly every prompt. The most dramatic gains came on prompts 4

and 7, where access to real market data allowed the model to give concrete, evidence-backed responses rather than generic advice.



2.3 Kennedy Wendl

I worked on frontend development and integrating the frontend with the backend APIs. Some of my main responsibilities included designing the UI of our application, maintaining styling consistency throughout, developing features, and resolving issues within our frontend code.

For Demo 1, I worked primarily with Dhvani on designing the overall layout and look of KittyCatQuant. Our client encouraged us to model our project after BunnyQuant, a stock trading platform with a variety of different features. I begin by sketching the

portfolio page and login screen to get a better understanding of how we wanted to create our application using React components and HTML and CSS styling. Then, using JavaScript, I built out the Buy and Sell Page. Originally, the Buy and Sell page contained only 5 stocks, but as our project progressed, this number grew to over 100. Throughout the semester, I continued to make adjustments to the page to account for the increase in stocks. Additionally, I maintained consistency throughout the application by adjusting the Signup/Login pages to match the theme of our application.

For Demo 2, I designed and built out the Stock News page as well as the predictive market insight. One of the major acceptance criteria our client provided us with was the ability for users to learn more about what factors may influence the stock market and, therefore, increase or decrease the price of certain stocks. Caleb was responsible for gathering and storing the news articles, while I connected to the backend to display and visualize the articles. Additionally, I created the search and favorite/delete news article features.

Finally, for Demo 3, I was responsible for investigating methods to predict stock prices. After Demo 2, our client suggested meeting one of the two graduate teams to combine or integrate their work on LLM's. I organized a meeting with Ayush Jha, whose project focused on improving RSI with an LLM. RSI stands for Relative Strength Index and measures the speed as well as magnitude of recent changes to the stock price to identify overbought or oversold conditions. Generally, an RSI score above 70 indicates an overbought asset, while an RSI score below 30 indicates an oversold asset. In other words, an individual should sell their shares if RSI is above 70 and buy shares below 30.

After meeting with him to discuss our work as well as his, I redesigned the predictive market insight from Demo 2 to calculate the RSI of a year's worth of stock history. I felt this metric better encompassed the scope of our project and gave the user a more in-depth insight into whether they should buy, sell, or hold. Additionally, I researched different machine learning models to determine which model could best predict the increase or decrease in stock price. Ultimately, I chose XGBoost for its effectiveness at forecasting price directions and analyzing historical data. The

challenge with most unsupervised models, such as XGBoost, is that it relies on tabular data. When writing the Python and C# code to build out this feature, I ran into a variety of issues creating endpoints in the backend for my training data. The predicted price that the XGBoost model returned was not an accurate representation of what the past prices for the stocks displayed. After discussing with the team and going over our client's requirements, I decided not to include this feature in our application and move forward only with RSI.

2.4 Dhvani Mistry

I worked on the complete frontend development and integrated the frontend with the backend APIs throughout the project. My responsibilities included UI/UX design, responsive layouts, frontend architecture, backend integration, and improving the overall user experience of the platform.

In Demonstration 1, the main emphasis was on creating the front-end structure of the application. I conducted my research about some popular stock trading applications to have a clearer idea about the layout designs and trading processes of these applications. Following this research, I started working on the appearance of KittyCatQuant.

The initial work included creating the authentication pages, such as login and sign-up screens, while maintaining a clean and professional design theme. After implementing authentication pages, I developed the frontend skeleton structure for the major application modules that would later be expanded upon. These included:

- Portfolio page
- Watchlist page
- Buy and Sell page
- Settings page

This stage was all about making reusable layouts, navigation, and styles for the app. As the development progressed, the frontend advanced from simple page structures

to an interactive trading dashboard. This stage developed the portfolio dashboard that contained:

- Portfolio value visualization graph
- Display of stocks owned by the user
- Watchlist functionality
- Buy and sell stock interface

I connected these frontend components to the backend APIs so that data could be dynamically retrieved and displayed. I also improved the stock and portfolio graphs by adding better chart rendering, hover interactions, tooltip displays for dates and prices, and time range filtering options such as 7 days, 1 month, and 1 year.

In Demo 2, I paid great attention to frontend cleaning and feature enhancement. One of the significant changes made was the replacement of the static sidebar used repetitively by the dynamic one that filled itself up on all pages automatically.

Based on client feedback, I designed and implemented a stock news page that retrieves financial news articles and allows users to favorite articles for future viewing. I also designed the AI chatbot interface and integrated WebSocket communication so users could interact with the AI assistant in real time.

Moreover, I added improvements to the settings page that would reflect real-time user settings as opposed to placeholders or pseudoproperties. The Learn Mode is one of the client-requested features that had to be incorporated. The reason being that the Learn Mode was meant to enhance user experience through a better understanding of the financial dashboard and trading terms. It must be noted that the client wanted to build an educational learning platform for its customers.

For Demo 3, my contributions were mainly related to fixing bugs, optimizing, and making the application more stable. I resolved many frontend and integration problems, such as:

- Disabling the use of the cryptoUUID library to allow the application to run successfully on the class server.

- Sort the chat history messages based on timestamps
- Ensure that chat history messages received were sent to the right active chat session
- Enhance responsiveness to support different sizes and resolutions

In addition to the above, I was responsible for improving the UI, ensuring it was responsive, and making sure the entire front-end experience was seamless. During the process, I regularly checked the interactions between the front-end and back-end and made improvements where necessary.

Chapter 4: Results

Our finished application achieved the goals and visions discussed with the client. We achieved four main objectives by building a realistic paper trading marketplace where users can manage portfolios and execute trades with real consequences, integrating live market data to ground every interaction in accurate pricing, designing the platform with an educational focus to help users build financial intuition, and developing and rigorously evaluating an LLM-powered trading assistant capable of delivering personalized, data-driven guidance.

4.1 Creating A Realistic Trading Marketplace

The first goal of this project was to accurately mimic the experience of buying and selling stocks to allow users to learn without putting anything at risk. This was achieved by creating a professional user interface with easy to use features. Users of KittyCatQuant are greeted with a home page that is emblematic of real trading interfaces, as well as all the features that would typically be available on those platforms.

Buying and selling is a simple process with real implications. Although the starting cash balance is large and not tied to real funds, making poor trading choices will have real repercussions on this balance. This provides users a consequence free sandbox to practice in while maintaining the serious nature of investing.

The availability of news articles helps to contribute to the feeling of real investing. Articles can be accessed and filtered into different categories, helping users easily find information relevant to their individual needs. Sentiment analysis features help users understand the impact of certain pieces of news, both in the specific sectors or companies it affects and the level of impact it will have.

Lastly, the vast array of different stocks that can be traded on KittyCatQuant allows users to experiment with many different styles of investment. Users can trade over 100 different stocks with capability to handle more with different API permissions.

KittyCatQuant successfully delivers a trading experience that feels authentic without carrying real financial risk, giving users the tools and context they need to develop genuine investing intuition. From the familiar interface and real market data to sentiment analysis and a diverse stock catalog, every feature was designed with the learner in mind.

4.2 Real Market Data Integration

Another focus of this project was to utilize real market data rather collected automatically to allow the application to function correctly after deployment without manual intervention. Our development focused on creating infrastructure to gather data on a set schedule, detect and handle gaps in historical data, have an adjustable date range, and detect market events like stock splits.

The Stock Data Backfill Runner system utilizes the Massive API as well as the PostgreSQL database to store and fill gaps in data for all stocks over the last two years. This, in conjunction with the Daily Stock Data Runner system, which updates the database with new data every day after market close, provides comprehensive data coverage that will remain comprehensive indefinitely. The stock data collected is used to record snapshots of portfolio value to easily track the value of portfolios over time. This data is used to construct graphs and compute percentage based loss and gain metrics, providing real data about a user's portfolio and investment choices.

Another service providing real world data is the News service, which utilizes the GNews API to provide real articles to users. This service uses a caching system to limit API requests, opting to query articles from the cache or database and only accessing the API if the article is not found in either of those sources.

KittyCatQuant's data infrastructure ensures the application remains accurate and self-sustaining long after deployment, pulling real market prices, portfolio metrics, and news articles without any manual intervention. Every number a user sees from stock prices to portfolio gains to news sentiment is grounded in live, automatically refreshed data.

4.3 Educational Focus

One of the biggest points of emphasis of this project was to create an environment that promotes learning and is inviting for users who are new to the stock market. This was accomplished through the collective result of many features that were developed over the course of the project.

LearnMode is the biggest contributing factor to the learning environment created by KittyCatQuant. Learn Mode is a toggleable option found in the sidebar that provides a short explanation of all features found on the website. Hovering the mouse over any data point or action found on the website will provide a one to two sentence description to help users understand the options they have. This removes the intimidation factor that often discourages new investors from engaging with trading platforms, allowing users to explore freely and build confidence at their own pace.

Another source of information is the AI Chatbot Assistant, KittyAI. The focus of this chatbot is to help guide users through making decisions relating to the market. Rather than simply executing trades, KittyCatQuant encourages users to think critically about their choices by providing personalized, data-driven context through KittyAI. Users can ask questions about specific stocks, their portfolio performance, or general market trends and receive responses grounded in real data rather than generic advice.

Together, these features reflect a core design philosophy: that the best way to learn investing is by doing it, with the right guidance available at every step. KittyCatQuant lowers the barrier to entry without sacrificing the depth that makes the experience genuinely educational.

4.4 LLM Integration and Evaluation

A major point of emphasis from our client was to integrate an LLM into the platform as a trading assistant to help new traders ease into the stock market, as well as to

evaluate its effectiveness, both in terms of its understanding of the market and its ability to help newer traders.

The chat infrastructure was built to support a full conversational experience. Users can create threads of messages that are stored and organized by timestamp, allowing KittyAI to maintain context across a conversation. Messages are sent and received through a WebSocket connection, providing a smooth, real-time chat experience that feels natural and responsive.

The LLM was given access to a suite of data tools that allow it to pull live market prices, portfolio information, and other relevant data on demand. This gives KittyAI the ability to provide responses that are specific to the user and grounded in real numbers, rather than offering generic market commentary. The system prompt was carefully tuned to encourage the model to be direct, cite evidence, and surface actionable research directions rather than hedging its answers.

To evaluate the effectiveness of these improvements, a rubric was developed and applied to a set of 10 prompts in two rounds of evaluation once before and once after the system prompt improvements and tool access were introduced. The rubric assessed whether responses included researchable items, specific data or evidence, user-specific context, and a direct answer to the question asked. The improved configuration more than doubled average scores across the prompt set, demonstrating that thoughtful prompt engineering and tool integration can dramatically improve the quality of LLM responses in a financial assistant context.

Ultimately, KittyAI represents more than just a chatbot it is a personalized guide that grows more useful as users engage with the platform, helping bridge the gap between raw market data and genuine financial understanding.